

Lab # 1

Client-side technologies: HTML-I

OBJECTIVE:

To understand the basic structure of HTML, its elements, and their applications for creating a personal site.

THEORY:

HTML (HyperText Markup Language) is the standard markup language used to create web pages. It provides the skeleton structure of the web document and allows developers to define the content and its arrangement.

1. The Anatomy of an HTML Tag:

HTML tags are the building blocks of web pages. Each tag is a pair of angled brackets enclosing an element's name.

2. What We're Building - HTML Personal Site:

A personal website typically contains the following:

- A homepage.
- Sections like biography, skills, and projects.
- Images and hyperlinks for navigation.

Example Code for a Simple Homepage:

```
<> abc.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  |   <title>My Personal Website</title>
5  </head>
6  <body>
7  |   <h1>Welcome to My Personal Site!</h1>
8  |   <p>Hello! I am [Your Name]. This is a simple webpage built using HTML.</p>
9  </body>
10 </html>
```

3. What is The HTML Boilerplate?

The HTML boilerplate refers to the basic template of an HTML file.

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9
10 </body>
11 </html>
```

4. HTML Lists

Lists are used to group related items. There are two main types:

- Ordered Lists (): Items are numbered.
- Unordered Lists (): Items use bullets.

```
1  <!-- ... -->
2
3  <!-- ... -->
4
5  <!-- ... -->
6
7  <!-- ... -->
8  <body>
9      <h3>My Favorite Fruits:</h3>
10     <ul>
11         <li>Apple</li>
12         <li>Banana</li>
13         <li>Cherry</li>
14     </ul>
15
16     <h3>Steps to Cook:</h3>
17     <ol>
18         <li>Preheat the oven.</li>
19         <li>Mix the ingredients.</li>
20         <li>Bake for 30 minutes.</li>
21     </ol>
22
23 </body>
24 </html>
```

5. HTML Image Elements

The `<img>` tag is used to add images to your website.

```

```

- `src`: Specifies the image source (file path or URL).
- `alt`: Provides alternative text if the image doesn't load.
- `width` and `height`: Control the dimensions of the image.

6. HTML Links and Anchor Tags

Links are created using the `<a>` tag.

```
<a href="https://example.com">Visit Example Website</a>
```

- `href`: Specifies the URL to link to.
- Link text is clickable.

LAB TASK:

1: Create a simple HTML webpage that serves as a personal homepage. The webpage should include a main heading with your name, a short paragraph introducing yourself, and a bulleted list of three hobbies or favorite activities. Additionally, add a hyperlink to a website you like and an image that represents one of your hobbies. Save the file as `homepage.html`.

2: Design a basic personal portfolio website consisting of two HTML pages. The first page should include a welcome message, a brief introduction, and a list of your top three favorite books or movies. The second page should contain a small gallery with three images of your choice and a heading above each image to describe it. Save the pages as `index.html` and `gallery.html`, and store them in a folder named `PortfolioWebsite`.

Lab # 2

Client-side technologies: HTML-II

OBJECTIVE

To learn how to create and use HTML tables for structuring data, use tables for layout design, type emojis, and create basic HTML forms.

THEORY

1: HTML Tables

HTML tables are used to display data in a structured format using rows and columns.

Basic Structure of an HTML Table:

```
8   <body>
9   |   <table>
10  |   |   <tr>
11  |   |   |   <th>Heading 1</th>
12  |   |   |   <th>Heading 2</th>
13  |   |   </tr>
14  |   |   <tr>
15  |   |   |   <td>Data 1</td>
16  |   |   |   <td>Data 2</td>
17  |   |   </tr>
18  |   </table>
19  |
20  </body>
21  </html>
```

- `<table>`: Defines the table.
- `<tr>`: Defines a table row.
- `<th>`: Defines a table header (bold and centered by default).
- `<td>`: Defines table data (regular cell).

Example

```
8   <body>
9       <table border="1">
10          <tr>
11              <th>Name</th>
12              <th>Age</th>
13              <th>City</th>
14          </tr>
15          <tr>
16              <td>Ali</td>
17              <td>22</td>
18              <td>Karachi</td>
19          </tr>
20          <tr>
21              <td>Ayesha</td>
22              <td>20</td>
23              <td>Lahore</td>
24          </tr>
25      </table>
26
27
28  </body>
```

Using HTML Tables for Layout

Tables can also be used to structure a webpage layout. While CSS is preferred for layouts, tables provide a simple way to create grid-based designs.

Example Layout Using Tables:

```
8   <body>
9       <table border="1" width="100%">
10          <tr>
11              <td colspan="2" align="center"><b>Header</b></td>
12          </tr>
13          <tr>
14              <td width="20%"><b>Sidebar</b></td>
15              <td>Main Content</td>
16          </tr>
17          <tr>
18              <td colspan="2" align="center"><b>Footer</b></td>
19          </tr>
20      </table>
21
22
23  </body>
```

HTML Tables Code Challenge

Try creating a timetable for your weekly classes using an HTML table. Include the following:

- Days of the week as rows.
- Subjects and time slots as columns.
- Use the <thead>, <tbody>, and <tfoot> tags to separate different sections of the table.

How to Type Emojis

Emojis can be added to an HTML document using their Unicode or by directly copying them.

Example:

```
8   <body>
9   |   <p>I ❤️ HTML! 🌟</p>
10  <p>Unicode Example: 😀 (😄)</p>
11  </body>
12  </html>
```

HTML Forms

Forms are a crucial part of web development that allow users to input data, which can then be sent to a server for processing. They are used for various purposes, such as user registration, login, feedback collection, online orders, and surveys. An HTML form consists of various form elements like text fields, checkboxes, radio buttons, dropdown menus, and submit buttons.

Key Attributes of the <form> Tag

- action: Specifies the URL where the form data will be sent.
- method: Defines how the data is sent to the server. Common methods are:
- GET: Appends form data to the URL; used for simple requests.
- POST: Sends form data in the HTTP body; used for sensitive or large data.
- enctype: Specifies the encoding type for the form data (e.g., multipart/form-data for file uploads).

Example of a Complete HTML Form

```

 9      <form action="/submit" method="POST">
10          <label for="firstname">First Name:</label>
11          <input type="text" id="firstname" name="firstname" required><br><br>
12
13          <label for="lastname">Last Name:</label>
14          <input type="text" id="lastname" name="lastname" required><br><br>
15
16          <label for="age">Age:</label>
17          <input type="number" id="age" name="age" min="1" max="100"><br><br>
18
19          <label for="gender">Gender:</label>
20          <input type="radio" id="male" name="gender" value="male"> Male
21          <input type="radio" id="female" name="gender" value="female"> Female<br><br>
22
23          <label for="hobbies">Hobbies:</label>
24          <input type="checkbox" name="hobby" value="sports"> Sports
25          <input type="checkbox" name="hobby" value="music"> Music<br><br>
26
27          <label for="country">Country:</label>
28          <select id="country" name="country">
29              <option value="pakistan">Pakistan</option>
30              <option value="india">India</option>
31          </select><br><br>
32
33          <label for="bio">Short Bio:</label><br>
34          <textarea id="bio" name="bio" rows="5" cols="30" placeholder="Tell us about yourself..."></textarea><br><br>
35
36          <input type="submit" value="Submit">
37      </form>

```

LAB TASK

Create a webpage that includes a timetable of your weekly classes using an HTML table. The table should have headings for each day of the week and rows for each time slot. Also, include a registration form below the timetable where users can enter their name, email, and select their class timing from a dropdown.

HOME TASK

Design a basic feedback form using HTML. The form should include fields for name, email, feedback message (textarea), and a rating system (radio buttons). Add a submit button at the end and save the file as feedback.html.

Lab # 3

Client-side technologies: CSS-I

OBJECTIVE

To understand the basics of CSS, its syntax, different types of stylesheets, and how to use selectors, classes, and IDs to style elements effectively.

THEORY

1: Introduction to CSS

CSS (Cascading Style Sheets) is used to control the style and layout of web pages. It separates content (HTML) from design (CSS), making it easier to maintain and update websites. CSS allows you to set properties like colors, fonts, margins, padding, and layouts.

2: The Anatomy of CSS Syntax

CSS follows a simple syntax:

- A CSS rule consists of a selector and a declaration block.
- The **selector** specifies which HTML element(s) the rule applies to.
- The **declaration block** contains one or more declarations inside curly braces { }.
- Each declaration includes a property and a value, separated by a colon : and ends with a semicolon ;.

Example Syntax:

```
css
selector {
    property: value;
    property: value;
}
```

3: Inline CSS, Internal CSS, and External CSS

1. **Inline CSS:** Styles are applied directly within an HTML element using the `style` attribute.

Example:

```
html
```

```
<p style="color: blue; font-size: 16px;">This is a blue  
paragraph.</p>
```

2. **Internal CSS:** Styles are written inside a `<style>` tag within the `<head>` section of the HTML document.

Example:

```
html  
<style>  
  p {  
    color: green;  
    font-size: 18px;  
  }  
</style>
```

3. **External CSS:** Styles are defined in a separate file and linked to the HTML document using a `<link>` tag.

Example:

- o style.css (External File):

```
css  
h1 {  
  color: red;  
  text-align: center;  
}
```

- o HTML File:

```
html  
  
<link rel="stylesheet" href="style.css">
```

4: CSS Selectors

Selectors are patterns used to select and style elements.

Basic Selectors:

- `*`: Universal selector; selects all elements.

```
css  
* {  
  margin: 0;  
  padding: 0;  
}
```

- `element`: Selects all elements of a specific type.

```
css
```

```
p {  
    color: gray;  
}
```

- **.class:** Selects all elements with a specific class.

```
css
```

```
.highlight {  
    background-color: yellow;  
}
```

- **#id:** Selects a specific element with an ID.

```
css
```

```
#main-header {  
    font-size: 24px;  
}
```

- **[attribute]:** Selects elements with a specific attribute.

```
css
```

```
input[type="text"] {  
    border: 1px solid black;  
}
```

5: Classes vs. IDs

- **Classes (class):** Used to apply styles to multiple elements.

Example:

Html

```
<p class="highlight">This paragraph is highlighted.</p>  
<p class="highlight">This one too!</p>
```

Css

```
.highlight {  
    background-color: yellow;  
}
```

- **IDs (id):** Used to apply styles to a single, unique element.

Example:**Html**

```
<h1 id="main-title">Welcome to CSS!</h1>
```

Css

```
#main-title {  
    color: navy;  
    font-size: 36px;  
}
```

LAB TASK

Design a homepage for a university that includes:

- A main heading styled using internal CSS to display the university name prominently.
- A navigation bar for links like "Home," "About Us," "Departments," and "Contact Us," styled using external CSS.
- A welcome paragraph styled using inline CSS to set a custom font size and background color.

HOME TASK

Create a personal portfolio webpage that includes:

- A main heading with your name, styled using an ID.
- A section for your hobbies, each with a subheading styled using classes.
- A list of your skills (ordered or unordered), with custom styles applied using an external stylesheet.

Lab # 4

Client-side technologies: CSS-II

OBJECTIVE

To explore advanced CSS concepts, including the Box Model, positioning, typography, and the role of divs in website styling.

THEORY

1: Favicons

Favicons are small icons associated with a webpage, displayed in the browser tab or bookmark. They enhance user experience and brand recognition.

To add a favicon to your webpage, use the following code inside the `<head>` section:

```
html
```

```
<link rel="icon" href="favicon.ico" type="image/x-icon">
```

2: Divs

The `<div>` element is a container used to group other HTML elements for styling and layout purposes. Divs are highly versatile and help structure a webpage into meaningful sections.

Example:

```
html
```

```
<div class="header">Header Section</div>
<div class="content">Main Content</div>
<div class="footer">Footer Section</div>
```

3: The Box Model of Website Styling

The CSS Box Model describes the structure of an HTML element. It consists of:

- **Content:** The inner text or HTML of the element.
- **Padding:** Space between the content and the border.
- **Border:** Surrounds the padding.
- **Margin:** Space between the border and other elements.

Example:

CSS

```
div {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 10px;  
}
```

4: CSS Display Property

The `display` property controls how elements are rendered on a page.

Common values include:

- `block`: The element takes up the entire width.
- `inline`: The element flows within a line.
- `flex`: Used for flexible layouts.

Example:

CSS

```
p {  
  display: inline;  
}  
div {  
  display: block;  
}
```

5: CSS Static, Relative, and Absolute Positioning

Positioning determines how elements are placed on a webpage:

- **Static:** Default positioning. Elements are placed in the normal flow of the document.
- **Relative:** Positioned relative to its normal position.
- **Absolute:** Positioned relative to the nearest positioned ancestor.

Example:

CSS

```
div.static {  
    position: static;  
}  
div.relative {  
    position: relative;  
    top: 10px;  
    left: 20px;  
}  
div.absolute {  
    position: absolute;  
    top: 50px;  
    left: 100px;  
}
```

6: CSS Typography

Typography focuses on styling text to improve readability and aesthetics. Common properties include:

- **Font-family:** Specifies the font.
 - **Font-size:** Sets the text size.
 - **Font-weight:** Adjusts the thickness of the font.
 - **Line-height:** Controls the spacing between lines of text.
- Example:

```
css  
  
p {  
    font-family: Arial, sans-serif;  
    font-size: 16px;  
    font-weight: bold;  
    line-height: 1.5;  
}
```

LAB TASK:

Design a **product landing page** that includes:

1. A favicon for the page.
2. A header, content section, and footer styled using **divs** and the **Box Model**.
3. Use **relative** positioning to adjust the content placement and **absolute** positioning for a floating "Contact Us" button.
4. Style the typography of the page (e.g., font family, size, and line spacing).

HOME TASK:

Design a homepage for a book store that includes:

1. A header section with the store name styled using CSS typography.
2. A section for featured books, divided into categories using divs.
3. Use the Box Model to add padding, borders, and margins to the book sections.
4. Include a "Buy Now" button for each book, styled using CSS positioning and typography.

Lab # 5

Client-side technologies: BOOTSTRAP

OBJECTIVE

To understand and implement Bootstrap, a popular front-end framework, for building responsive and modern web designs.

THEORY

What is Bootstrap?

Bootstrap is an open-source CSS framework designed to simplify web development. It provides pre-designed components like navigation bars, buttons, modals, and forms, along with a responsive grid system to create mobile-friendly layouts. It helps developers save time by reducing the need to write extensive CSS.

1: Installing Bootstrap

You can include Bootstrap in your project in two ways:

1. **Using CDN:** Add the following `<link>` to the `<head>` section of your HTML file:

```
html

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/b
ootstrap.min.css" rel="stylesheet">
```

2. **Downloading Bootstrap:**

Download the Bootstrap files from getbootstrap.com, and include them in your project directory.

2: Web Design - Wire Framing

Wireframing is the process of sketching the layout and structure of a website before coding. It helps visualize the placement of elements like headers, navigation bars, and content sections. Tools like Figma, Adobe XD, or even paper sketches are commonly used for wireframing.

3: The Bootstrap Navigation Bar

Bootstrap's navigation bar component provides a responsive and customizable way to create menus. Here's an example:

html

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">MySite</a>
    <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav">
        <li class="nav-item"><a class="nav-link"
href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link"
href="#">About</a></li>
        <li class="nav-item"><a class="nav-link"
href="#">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>
```

4: Bootstrap Grid Layout System

The grid system is the foundation of Bootstrap layouts. It uses rows and columns to create responsive layouts. A row can have up to 12 columns, and you can specify how many columns an element should span.

Example:

html

```
<div class="container">
  <div class="row">
    <div class="col-md-6">Left Column</div>
    <div class="col-md-6">Right Column</div>
  </div>
</div>
```

5: Adding Grid Layouts to Our Website

Use the grid system to structure your website's sections, such as headers, content areas, and footers, while ensuring they adapt to various screen sizes.

6: Bootstrap Containers

Bootstrap provides two types of containers to manage layouts:

1. **.container:** Fixed width and responsive.
2. **.container-fluid:** Full width across all screen sizes.

Example:

html

```
<div class="container">
  <h1>Fixed Container</h1>
</div>
<div class="container-fluid">
  <h1>Fluid Container</h1>
</div>
```

7: Bootstrap Buttons & Font Awesome

Bootstrap includes pre-styled buttons, which can be customized using classes like `.btn-primary`, `.btn-success`, and `.btn-danger`. Font Awesome is an icon library often used alongside Bootstrap for adding icons.

Example:

html

```
<button class="btn btn-primary">Click Me</button>
<i class="fa fa-check"></i>
```

LAB TASK

Create a responsive webpage for a **Tech Startup**, including:

1. A **navigation bar** with links to Home, Services, and Contact Us.
2. A **hero section** using the **Bootstrap grid system** with a heading, subheading, and a "Learn More" button styled using Bootstrap buttons.
3. Use a **container-fluid** to include a list of services, each in its own column.

HOME TASK

Design a responsive webpage for a **Travel Agency**, including:

1. A **navigation bar** with a logo and links to Home, Packages, and Contact Us.
2. Use the **grid layout system** to display three featured travel destinations with images and descriptions.
3. Add **Font Awesome icons** for contact information (e.g., phone, email).

Lab # 6

Client-side technologies: Javascript - I

Objective

To learn the fundamentals of JavaScript, including its data types, variables, control statements, and functions, and how it adds dynamic behavior to websites.

Theory

JavaScript is a powerful client-side scripting language that enables interactive and dynamic features on web pages. It is widely used for tasks such as form validation, creating animations, and handling user events. JavaScript can be embedded within an HTML document or included as an external file.

JavaScript Alerts - Adding Behavior to Websites

JavaScript alerts are used to display messages to users. The `alert()` function is often used for debugging or providing basic notifications to users.

Example:

```
javascript  
  
alert("Welcome to JavaScript!");
```

Data Types

JavaScript has several data types, including:

- **String:** Text, e.g., "Hello, world!"
- **Number:** Numerical values, e.g., 42 or 3.14
- **Boolean:** Logical values, true or false
- **Object:** Collection of properties, e.g., { name: "John", age: 25 }
- **Array:** List of values, e.g., [1, 2, 3]
- **Null and Undefined:** Special data types representing empty or uninitialized values.

JavaScript Variables

Variables store data values. Use `var`, `let`, or `const` to declare variables.

Example:

```
javascript

let name = "Alice"; // A variable that can change
const pi = 3.14;    // A constant value
```

String Functions

JavaScript provides many functions to manipulate strings, such as:

- `length`: Returns the length of a string.
- `toUpperCase()`: Converts a string to uppercase.
- `toLowerCase()`: Converts a string to lowercase.

Example:

```
javascript

let text = "JavaScript";
console.log(text.length);           // Output: 10
console.log(text.toUpperCase());    // Output: "JAVASCRIPT"
```

Basic Arithmetic and the Modulo Operator

JavaScript supports arithmetic operators like `+`, `-`, `*`, `/`, and `%` (modulo).

Example:

```
javascript

let a = 10, b = 3;
console.log(a % b); // Output: 1 (remainder of 10 / 3)
```

Increment and Decrement Expressions

Increment (`++`) and decrement (`--`) operators increase or decrease a value by 1.

Example:

```
javascript

let x = 5;
x++; // x becomes 6
x--; // x becomes 5
```

Control Statements: Using If-Else Conditionals

if-else statements control program flow based on conditions.

Example:

```
javascript

let score = 85;
if (score > 50) {
  console.log("You passed!");
} else {
  console.log("You failed!");
}
```

Control Statements: While Loops

A while loop runs as long as a condition is true.

Example:

```
javascript

let count = 1;
while (count <= 5) {
  console.log(count); // Outputs 1, 2, 3, 4, 5
  count++;
}
```

Control Statements: For Loops

A for loop is ideal for iterating a specific number of times.

Example:

```
javascript

for (let i = 1; i <= 5; i++) {
  console.log(i); // Outputs 1, 2, 3, 4, 5
}
```

JavaScript Objects and Functions

Objects in JavaScript are collections of properties and methods. Functions define reusable blocks of code.

Example of an object and a function:

```
javascript

let student = {
  name: "John",
  age: 22,
  greet: function () {
    console.log(`Hello, my name is ${this.name}.`);
  }
};
student.greet(); // Output: "Hello, my name is John."

function add(a, b) {
  return a + b;
}
console.log(add(5, 10)); // Output: 15
```

Lab Task

Design a webpage using JavaScript to perform the following:

1. Display an alert message welcoming users to your webpage.
2. Create a form to input a user's name and display a personalized greeting using JavaScript.
3. Implement a `for` loop to display the first 10 natural numbers in the console.

Home Task

Create a simple JavaScript program to:

1. Ask the user for a number input using `prompt()`.
2. Check whether the number is even or odd using an `if-else` statement.
3. Display the result in an alert box.

Lab # 7

Client-side technologies: Javascript - II

Objective

To understand how JavaScript interacts with HTML and CSS using the Document Object Model (DOM), and how to manipulate webpage elements dynamically.

Theory

The **Document Object Model (DOM)** is a programming interface for web documents. It represents the structure of a web page as a tree of objects, where each node represents a part of the page (e.g., elements, attributes, text). JavaScript can interact with the DOM to manipulate HTML and CSS elements dynamically, allowing for interactive web pages.

In this lab, you will learn how JavaScript can:

- Access HTML elements using the DOM.
- Modify elements dynamically.
- Change CSS styles via JavaScript.
- Respond to user actions by manipulating the DOM.

What is the DOM?

The DOM is essentially a representation of the HTML document as a tree structure. Each element in the HTML document is an object in the tree, and JavaScript can access and modify these elements through the DOM.

For example, in the HTML structure:

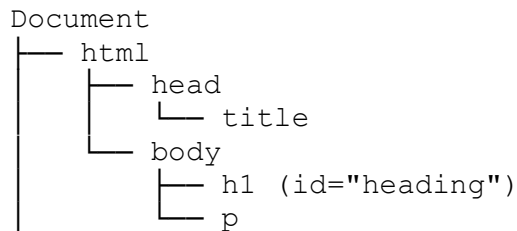
html

```
<!DOCTYPE html>
<html>
  <head>
    <title>DOM Example</title>
  </head>
  <body>
    <h1 id="heading">Hello, DOM!</h1>
    <p>Welcome to learning JavaScript and DOM.</p>
  </body>
```

```
</html>
```

The DOM representation might look like this:

```
less
```



Interacting with the DOM Using JavaScript

Accessing DOM Elements

JavaScript can access HTML elements using methods like:

- `getElementById()`: Access elements by their `id`.
- `getElementsByTagName()`: Access elements by their tag name.
- `getElementsByClassName()`: Access elements by their class name.
- `querySelector()`: Select the first element matching a CSS selector.
- `querySelectorAll()`: Select all elements matching a CSS selector.

Example: Accessing Elements

```
javascript
```

```
// Access an element by its ID
let heading = document.getElementById('heading');
console.log(heading); // Output: <h1 id="heading">Hello, DOM!</h1>

// Access all <p> elements by tag name
let paragraphs = document.getElementsByTagName('p');
console.log(paragraphs); // Output: [<p>Welcome to learning JavaScript
and DOM.</p>]
```

Manipulating DOM Elements

Once you access an element, you can manipulate its content, attributes, or style.

Changing Content

To change the content of an element, you can modify the `innerHTML` property.

```
javascript
```

```
// Change the text content of the heading
```

```
heading.innerHTML = "JavaScript DOM Manipulation";
```

Changing Styles

You can change the CSS styles of an element by modifying its `style` property.

```
javascript

// Change the color and font size of the heading
heading.style.color = "blue";
heading.style.fontSize = "30px";
```

Changing Attributes

To modify attributes like `src`, `href`, or `class`, you can use the `setAttribute()` method.

```
javascript

// Change the "href" attribute of a link
let link = document.getElementById('myLink');
link.setAttribute("href", "https://www.example.com");
```

Creating New Elements

You can also create new elements and append them to the document.

```
javascript

// Create a new paragraph
let newParagraph = document.createElement("p");
newParagraph.innerHTML = "This is a new paragraph created by
JavaScript.";

// Append the new paragraph to the body
document.body.appendChild(newParagraph);
```

Removing Elements

To remove an element, you can use the `removeChild()` method.

```
javascript

// Remove the first paragraph
document.body.removeChild(paragraphs[0]);
```

Event Handling

JavaScript allows you to respond to user interactions using events like `click`, `mouseover`, `keydown`, etc. You can attach event listeners to DOM elements.

```
javascript
```

```
// Add a click event to the heading
heading.addEventListener("click", function() {
  alert("Heading clicked!");
});
```

DOM Example: Interactive Webpage

Here's a full example that demonstrates how to use JavaScript with HTML and CSS through the DOM:

HTML:

```
<body>
  <h1 id="heading">Click the Button to Change Content</h1>
  <p id="text">This paragraph will be updated by JavaScript.</p>
  <button id="myButton">Click Me</button>
</body>
```

CSS:

```
<style>
  #myButton {
    padding: 10px 20px;
    background-color: lightblue;
    border: none;
    cursor: pointer;
  }

  #myButton:hover {
    background-color: lightgreen;
  }
</style>
```

JAVASCRIPT:

```
<script>
  // Accessing elements
  let heading = document.getElementById("heading");
  let paragraph = document.getElementById("text");
  let button = document.getElementById("myButton");

  // Change content when button is clicked
  button.addEventListener("click", function() {
    heading.innerHTML = "Content Changed!";
    paragraph.innerHTML = "You have successfully clicked the button and changed the content.";
    paragraph.style.color = "green";
  });
</script>
```

OUTPUT (BEFORE CLICKING):

Click the Button to Change Content

This paragraph will be updated by JavaScript.

Click Me

OUTPUT (AFTER CLICKING):

Content Changed!

You have successfully clicked the button and changed the content.

Click Me

Lab Task:

1. Create a webpage with a heading, a paragraph, and a button.
2. Use JavaScript to change the content of the paragraph when the button is clicked.
3. Modify the button's style using JavaScript when the mouse hovers over it (using `mouseover` and `mouseout` events).
4. Add a new paragraph element dynamically when the user clicks the button.

Home Task:

1. Design a webpage with at least three sections: a header, content area, and footer.
2. Use JavaScript to dynamically update the header text when a button is clicked.
3. Allow the user to change the background color of the content area by selecting a color from a dropdown menu.
4. Use JavaScript to add a new footer element dynamically after 5 seconds of loading the page.